👤 1. Autoryzacja i Użytkownicy

- **POST /api/rejestracja/**
  - **Do czego służy:** Tworzenie nowego konta (Student lub Profesor).
  - **Co robi:** Waliduje dane (np. wymusza @student dla studentów, sprawdza unikalność maila) i bezpiecznie szyfruje hasło w bazie. Zwraca informacje o utworzonym koncie lub listę błędów, co poszło nie tak.
- **POST /api/logowanie/**
  - **Do czego służy:** Generowanie "biletu wstępu" (Tokena).
  - **Co robi:** Przyjmuje username i password. Jeśli są poprawne, wydaje unikalny klucz w formacie {"token": "xyz123..."}. Zespół frontendowy musi ten klucz zapisać u siebie (np. w localStorage).

💻 2. Zarządzanie Kodami (Magia ViewSetu)

Te endpointy zostały wygenerowane automatycznie na bazie Twojego modelu. Zwróć uwagę frontendowcom na to, jakich **metod HTTP** muszą tu używać.

- **GET /api/kody/**
  - **Do czego służy:** Pobieranie wszystkich kodów.
  - **Co robi:** Zwraca tablicę JSON ze wszystkimi snippetami zapisanymi w bazie. Tego użyją do zbudowania strony głównej (listy kodów). Na ten moment nie wymaga tokena.
- **POST /api/kody/**
  - **Do czego służy:** Dodawanie nowego kodu.
  - **Co robi:** Przyjmuje JSON z polami title, code, language. **Ważne:** Przekaż im, że od teraz muszą wysyłać to zapytanie z nagłówkiem Authorization: Token <ich_token>, inaczej serwer ich odrzuci!
- **GET /api/kody/{id}/** (np. /api/kody/12/)
  - **Do czego służy:** Pobieranie szczegółów jednego, konkretnego kodu.
  - **Co robi:** Jeśli użytkownik kliknie na liście w kod o ID 12, front wysyła zapytanie pod ten adres, a Ty zwracasz mu tylko ten jeden wpis.
- **PUT lub PATCH /api/kody/{id}/**
  - **Do czego służy:** Edytowanie istniejącego kodu.

    - **Co robi:** PUT nadpisuje cały kod, a PATCH pozwala zmienić tylko wybrany fragment (np. sam tytuł). To również będzie zabezpieczone tokenem.
- **DELETE /api/kody/{id}/**
    - **Do czego służy:** Całkowite usuwanie kodu z bazy.
    - **Co robi:** Wymaga dodania tokena w nagłówku.